

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CƠ BẢN I

THỰC TẬP CƠ SỞ



BÁO CÁO TUẦN

Đề tài:

Xây dựng website tuyển dụng việc làm

Giảng viên hướng dẫn	: Kim Ngọc Bách
Họ và tên sinh viên	: Đỗ Bình Minh
Ngày sinh	: 24/12/2005
Số điện thoại	: 0329279863
Mã sinh viên	: B23DCCN542
Lớp	: D23CQCN10-B
Nhóm lớp	: 11

Hà Nội – 2026

Mục lục

1. Tóm tắt các công việc đã thực hiện trong tuần	3
2. Các công việc đã thực hiện trong tuần	4
2.1. Trang danh sách công ty.....	4
2.1.1. Xử lý bên phía Frontend.....	4
2.1.2. Kết quả.	6
2.2. Tính năng phân trang.....	7
2.2.1. Xử lý bên phía Frontend.....	7
2.2.2. Xử lý bên phía backend.....	10
2.2.3. Kết quả.	13
2.3. Trang chi tiết công ty	14
2.3.1. Xử lý bên phía Frontend.....	14
2.3.2. Xử lý bên phía backend.....	16
2.3.3. Kết quả	18
2.4. Trang quản lý CV	19
2.4.1. Xử lý bên phía Frontend.....	19
2.4.2. Xử lý bên phía backend.....	20
2.4.3. Kết quả	22
3. Kết quả đạt được	22
4. Khó khăn gặp phải.....	23
5. Hướng xử lý.....	24
6. Kế hoạch tuần tiếp theo	25

1. Tóm tắt các công việc đã thực hiện trong tuần

Trong tuần này, em tiếp tục phát triển website tuyển dụng việc làm với các chức năng liên quan đến phần công ty và quản lý CV cho nhà tuyển dụng.

Đầu tiên, em xây dựng trang danh sách công ty ở phía frontend. Trang này có nhiệm vụ gọi API lấy danh sách công ty từ backend, lưu dữ liệu vào state và hiển thị từng công ty thông qua component CardCompanyItem. Nhờ đó, người dùng có thể xem được các công ty đang có trên hệ thống.

Tiếp theo, em bổ sung chức năng phân trang cho danh sách công ty. Ở phía frontend, em sử dụng state để lưu trang hiện tại và tổng số trang. Khi người dùng thay đổi trang, hệ thống sẽ gọi lại API với tham số page tương ứng để lấy dữ liệu của trang đó. Ở phía backend, em xử lý phân trang bằng cách lấy giá trị page từ query, tính số bản ghi cần bỏ qua bằng skip, giới hạn số công ty mỗi trang bằng limitItems và trả về tổng số trang cho frontend.

Bên cạnh đó, em xây dựng trang chi tiết công ty. Frontend lấy slug từ URL, gọi API chi tiết công ty và hiển thị các thông tin như logo, tên công ty, địa chỉ, mô hình công ty, số lượng nhân viên, thời gian làm việc, mô tả công ty và danh sách công việc thuộc công ty đó. Với phần mô tả công ty, em sử dụng dangerouslySetInnerHTML để render nội dung HTML trả về từ API, giúp giữ nguyên định dạng mô tả.

Ngoài ra, em phát triển trang quản lý CV dành cho nhà tuyển dụng. Frontend gọi API /company/cv/list, gửi kèm cookie để backend xác thực tài khoản công ty đang đăng nhập. Sau khi lấy dữ liệu thành công, danh sách CV được lưu vào state và hiển thị trên giao diện. Các giá trị như vị trí ứng tuyển, hình thức làm việc và trạng thái CV được đối chiếu với các danh sách cấu hình để hiển thị thành nội dung dễ hiểu như “Chưa duyệt”, “Đã duyệt”, “Từ chối”.

Ở phía backend, em xây dựng API lấy danh sách CV của công ty. Trước khi xử lý, hệ thống sử dụng middleware xác thực verifyTokenCompany để đảm bảo chỉ tài khoản công ty đã đăng nhập mới có thể xem danh sách CV. Sau đó, backend lấy ID công ty từ tài khoản đăng nhập, tìm các công việc thuộc công ty đó, lấy danh sách CV ứng tuyển vào các công việc tương ứng và trả dữ liệu về frontend.

2. Các công việc đã thực hiện trong tuần

2.1. Trang danh sách công ty

2.1.1. Xử lý bên phía Frontend

```
// eslint-disable @typescript-eslint/no-explicit-any
import { CardCompanyItem } from "@app/components/card/CardCompanyItem"
import { useEffect, useState } from "react";

export const CompanyList = () => {
  const [companyList, setCompanyList] = useState<any[]>([]);
  const [page, setPage] = useState(1);
  const [totalPage, setTotalPage] = useState(0);

  useEffect(() => {
    fetch(`${process.env.NEXT_PUBLIC_API_URL}/company/list?limitItems=2&page=${page}`)
      .then(res => res.json())
      .then(data => {
        if (data.code == "success") {
          setCompanyList(data.companyList);
          setTotalPage(data.totalPage);
        }
      })
  }, [page]);

  const handlePagination = (event: any) => {
    const value = event.target.value;
    setPage(parseInt(value));
  }

  return (
    <>
      <div className="grid lg:grid-cols-3 grid-cols-2 sm:gap-[20px] gap-x-[10px] gap-y-[20px]">
        {companyList.map((item) => (
          <CardCompanyItem key={item.id} {...item} />
        ))}
      </div>

      <div className="mt-[30px]">
        <select
          name=""
          className="border border-[#DEDED8] rounded-[8px] py-[12px] px-[18px] font-[400] text-[16px] text-[#414042]"
          onChange={handlePagination}>
          <Array(totalPage).fill("").map((item, index) => (
            <option key={index} value={index+1}>
              Trang {index+1}
            </option>
          ))>
        </select>
      </div>
    </>
  )
}
```

Code này là component **CompanyList** dùng để lấy danh sách công ty từ API rồi hiển thị ra giao diện.

"use client"

Dòng này dùng trong Next.js để báo component chạy phía client, vì bên dưới có dùng useState, useEffect.

```
const [companyList, setCompanyList] = useState<any[]>([]);
```

Tạo state companyList để lưu danh sách công ty lấy từ API. Ban đầu là mảng rỗng.

```
useEffect(() => {  
  fetch(`${process.env.NEXT_PUBLIC_API_URL}/company/list?limitItems=2`)
```

Khi component được render lần đầu, nó gọi API lấy danh sách công ty.
limitItems=2 nghĩa là chỉ lấy 2 công ty.

```
.then(res => res.json())  
.then(data => {  
  if(data.code === "success") {  
    setCompanyList(data.companyList);  
  }  
})
```

Sau khi API trả về dữ liệu JSON, nếu data.code là "success" thì lưu data.companyList vào state.

```
companyList.map((item) => (  
  <CardCompanyItem key={item.id} {...item} />  
))
```

Duyệt từng công ty trong companyList, rồi truyền dữ liệu công ty đó vào component CardCompanyItem.

```
{...item}
```

Nghĩa là truyền toàn bộ thuộc tính của item sang CardCompanyItem.

Ví dụ item có:

```
{  
  id: 1,  
  name: "ABC",  
  logo: "abc.png"  
}
```

thì nó tương đương:

```
<CardCompanyItem id={1} name="ABC" logo="abc.png" />
```

Phần cuối:

```
<select>  
  <option>Trang 1</option>  
  <option>Trang 2</option>  
  <option>Trang 3</option>  
</select>
```

Đây là ô chọn trang, nhưng hiện tại mới chỉ hiển thị giao diện, chưa có onChange nên chọn trang chưa làm gì cả.

2.1.2. Kết quả.

28.ITJobs**Đăng Nhập / Đăng Ký** 

Nhà tuyển dụng hàng đầu



VietQR

Hà Nội  3 Việc làm



FPT

Hà Nội  1 Việc làm

Trang 1 

**28.ITJobs**

Danh sách công ty đã được hiển thị trên trang.

2.2. Tính năng phân trang

2.2.1. Xử lý bên phía Frontend

```
export const CompanyList = () => {  
  const [companyList, setCompanyList] = useState<any[]>([]);  
  const [page, setPage] = useState(1);  
  const [totalPage, setTotalPage] = useState(0);  
  
  useEffect(() => {  
    fetch(`${process.env.NEXT_PUBLIC_API_URL}/company/list?limitItems=2&page=${page}`)  
      .then(res => res.json())  
      .then(data => {  
        if(data.code == "success") {  
          setCompanyList(data.companyList);  
          setTotalPage(data.totalPage);  
        }  
      })  
  }, [page]);  
  
  const handlePagination = (event: any) => {  
    const value = event.target.value;  
    setPage(parseInt(value));  
  }  
}
```

Đoạn code này đang thêm chức năng **phân trang** cho danh sách công ty.

```
const [companyList, setCompanyList] = useState<any[]>([]);
```

Lưu danh sách công ty lấy từ API.

```
const [page, setPage] = useState(1);
```

Lưu trang hiện tại. Ban đầu là **trang 1**.

```
const [totalPage, setTotalPage] = useState(0);
```

Lưu tổng số trang mà API trả về.

Phần gọi API:

```
useEffect(() => {
```

```
  fetch(`${process.env.NEXT_PUBLIC_API_URL}/company/list?limitItems=2&page=${page}`)  
    .then(res => res.json())  
    .then(data => {  
      if(data.code == "success") {  
        setCompanyList(data.companyList);  
        setTotalPage(data.totalPage);  
      }  
    })  
}, [page]);
```

Mỗi lần component chạy hoặc page thay đổi, nó sẽ gọi API lại.

Ví dụ:

```
/company/list?limitItems=2&page=1  
/company/list?limitItems=2&page=2  
/company/list?limitItems=2&page=3
```

limitItems=2 nghĩa là mỗi trang lấy 2 công ty.

Phần xử lý dữ liệu trả về:

```
if(data.code === "success") {  
  setCompanyList(data.companyList);  
  setTotalPage(data.totalPage);  
}
```

Nếu API trả về thành công thì:

```
setCompanyList(data.companyList);
```

Cập nhật danh sách công ty của trang hiện tại.

```
setTotalPage(data.totalPage);
```

Lưu tổng số trang.

Phần này rất quan trọng:

```
}, [page]);
```

Nghĩa là: **mỗi khi page thay đổi thì useEffect chạy lại.**

Ví dụ chọn sang trang 2:

```
setPage(2);
```

thì API sẽ tự gọi lại với:

```
page=2
```

Hàm đổi trang:

```
const handlePagination = (event: any) => {  
  const value = event.target.value;  
  setPage(parseInt(value));  
}
```

Khi người dùng chọn trang trong <select>, hàm này lấy giá trị trang rồi cập nhật vào state page.

```

<div className="mt-[30px]">
  <select
    name=""
    className="border border-[#DEDEDE] rounded-[8px] py-[12px] px-[18px] font-[400] text-[16px] text-[#414042]"
    onChange={handlePagination}
  >
    {Array(totalPage).fill("").map((item, index) => (
      <option key={index} value={index+1}>
        Trang {index+1}
      </option>
    ))}
  </select>
</div>

```

Đoạn này là phần **hiển thị ô chọn trang** dựa trên totalPage.

```

<select
  name=""
  className="..."
  onChange={handlePagination}
>

```

select là ô dropdown chọn trang.

onChange={handlePagination}

Khi người dùng chọn trang khác, hàm handlePagination sẽ chạy.

Ví dụ chọn **Trang 2** thì hàm nhận được:

```
event.target.value = "2"
```

Sau đó trong hàm trước đó:

```
setPage(parseInt(value));
```

nó đổi "2" thành số 2, rồi cập nhật page.

Phần quan trọng nhất là đoạn này:

```

{Array(totalPage).fill("").map((item, index) => (
  <option key={index} value={index + 1}>
    Trang {index + 1}
  </option>
))}

```

Nó dùng để tạo danh sách option tự động theo tổng số trang.

Ví dụ:

```
totalPage = 3
```

thì:

Array(totalPage)

sẽ tạo ra mảng có 3 phần tử.

Sau đó:

.fill("")

đổ dữ liệu rỗng vào mảng để có thể dùng .map().

Rồi .map() sẽ chạy 3 lần:

index = 0 → Trang 1

index = 1 → Trang 2

index = 2 → Trang 3

Kết quả HTML tương đương:

<option value={1}>Trang 1</option>

<option value={2}>Trang 2</option>

<option value={3}>Trang 3</option>

2.2.2. Xử lý bên phía backend

```
let page = 1;
if(req.query.page && parseInt(`${req.query.page}`) > 0) {
  page = parseInt(`${req.query.page}`);
}
const skip = (page - 1) * limitItems;
const totalRecord = await Job.countDocuments(find);
const totalPage = Math.ceil(totalRecord/limitItems);
```

```
res.json({
  code: "success",
  message: "Thành công",
  companyList: companyListFinal,
  totalPage: totalPage
})
```

Đoạn này là **code backend xử lý phân trang**.

let page = 1;

Mặc định nếu frontend không gửi trang nào lên thì backend hiểu là đang ở **trang 1**.

```
if(req.query.page && parseInt(`${req.query.page}`) > 0) {
  page = parseInt(`${req.query.page}`);
}
```

Kiểm tra trên URL có truyền page không.

Ví dụ frontend gọi:

/company/list?limitItems=2&page=3

thì:

req.query.page

sẽ bằng "3".

Sau đó dùng parseInt() để đổi từ chuỗi "3" sang số 3.

const skip = (page - 1) * limitItems;

Tính số bản ghi cần bỏ qua.

Ví dụ limitItems = 2:

page skip Ý nghĩa

1 0 lấy từ bản ghi đầu tiên

2 2 bỏ qua 2 bản ghi đầu

3 4 bỏ qua 4 bản ghi đầu

Công thức:

skip = (page - 1) * limitItems

Ví dụ trang 3:

skip = (3 - 1) * 2 = 4

Tức là bỏ qua 4 công ty đầu, rồi lấy tiếp 2 công ty.

const totalRecord = await Job.countDocuments(find);

Đếm tổng số bản ghi phù hợp với điều kiện find.

Ví dụ có tổng cộng 7 công ty thì:

totalRecord = 7

const totalPages = Math.ceil(totalRecord / limitItems);

Tính tổng số trang.

Ví dụ:

`totalRecord = 7`

`limitItems = 2`

thì:

$7 / 2 = 3.5$

`Math.ceil(3.5) = 4`

Nghĩa là cần **4 trang**.

Phần trả dữ liệu về frontend:

```
res.json({  
  code: "success",  
  message: "Thành công",  
  companyList: companyListFinal,  
  totalPage: totalPage  
})
```

Backend trả về:

`companyList`

là danh sách công ty của trang hiện tại.

`totalPage`

là tổng số trang để frontend tạo dropdown:

Trang 1

Trang 2

Trang 3

...

Tóm lại: backend nhận page từ URL, tính skip, lấy dữ liệu đúng trang, tính totalPage, rồi trả về cho frontend hiển thị phân trang.

2.2.3. Kết quả.

28.ITJobs

Đăng Nhập / Đăng Ký

Nhà tuyển dụng hàng đầu



VietQR

Hà Nội 3 Việc làm



FPT

Hà Nội 1 Việc làm

Trang 1

Trang 1

Trang 2



28.ITJobs

Tính năng phân trang đã hiển thị theo số công ty

13

2.3. Trang chi tiết công ty

2.3.1. Xử lý bên phía Frontend

```
export default async function CompanyDetailPage({ params }: {
  params: {
    slug: string
  }
}) {
  const { slug } = await params;
  const res = await fetch(`${process.env.NEXT_PUBLIC_API_URL}/company/detail/${slug}`);
  const data = await res.json();

  let companyDetail: any = null;
  let jobList: any = null;
  if(data.code == "success") {
    companyDetail = data.companyDetail;
    jobList = data.jobs;
  }
}
```

Đoạn này là code trang **chi tiết công ty** trong Next.js.

```
export default async function CompanyDetailPage({ params }: {
  params: {
    slug: string
  }
})
```

Đây là một page component dạng async, dùng để lấy dữ liệu trước khi render trang.

params.slug thường lấy từ URL động.

Ví dụ đường dẫn:

/company/fpt-software

thì:

slug = "fpt-software"

const { slug } = await params;

Lấy slug từ params.

Nếu bạn dùng Next.js 15 thì viết kiểu này được.

Còn nếu Next.js cũ hơn, thường viết:

const { slug } = params;

```
const res = await  
fetch(`${process.env.NEXT_PUBLIC_API_URL}/company/detail/${slug}`);
```

Gọi API lấy chi tiết công ty theo slug.

Ví dụ:

```
/company/detail/fpt-software
```

```
const data = await res.json();
```

Chuyển dữ liệu API trả về sang JSON.

```
let companyDetail: any = null;
```

```
let jobList: any = null;
```

Tạo 2 biến ban đầu là null.

Trong đó:

companyDetail

dùng để lưu thông tin chi tiết công ty.

jobList

dùng để lưu danh sách việc làm của công ty đó.

```
if(data.code === "success") {  
  companyDetail = data.companyDetail;  
  jobList = data.jobs;  
}
```

Nếu API trả về thành công thì gán dữ liệu:

```
companyDetail = data.companyDetail;
```

lưu thông tin công ty.

```
jobList = data.jobs;
```

lưu danh sách job của công ty.

Tóm lại: đoạn code này lấy slug từ URL, gọi API chi tiết công ty theo slug, sau đó lưu thông tin công ty và danh sách job để hiển thị ra giao diện.

```
<div className="border border-[#DEDEDE] rounded-[8px] p-[20px] mt-[20px]">  
  <div dangerouslySetInnerHTML={{ __html: companyDetail.description }} />  
</div>
```

Đây là đoạn code dùng để hiển thị thông tin mô tả chi tiết của công ty. Dữ liệu mô tả được lấy từ `companyDetail.description` và được render dưới dạng HTML thông qua `dangerouslySetInnerHTML`, giúp giữ nguyên định dạng nội dung như đoạn văn, danh sách hoặc tiêu đề nếu dữ liệu trả về từ API có chứa thẻ HTML.

2.3.2. Xử lý bên phía backend

```
router.get(
  "/list",
  companyController.list
)
```

Đoạn code trên khai báo route `/list` cho chức năng lấy danh sách công ty. Khi phía client gửi yêu cầu GET đến API này, hệ thống sẽ chuyển yêu cầu sang hàm `companyController.list` để xử lý việc truy vấn dữ liệu công ty, phân trang nếu có, sau đó trả kết quả về cho frontend hiển thị.

```
export const detail = async (req: Request, res: Response) => {
  try {
    const id = req.params.id;
    const record = await AccountCompany.findOne({
      _id: id
    })

    if(!record) {
      res.json({
        code: "error",
        message: "Dữ liệu không hợp lệ!"
      })
      return;
    }

    const companyDetail = {
      id: record.id,
      logo: record.logo,
      companyName: record.companyName,
      address: record.address,
      companyModel: record.companyModel,
      companyEmployees: record.companyEmployees,
      workingTime: record.workingTime,
      workOvertime: record.workOvertime,
      description: record.description,
    };

    // Danh sách công việc của công ty
    const dataFinal = [];

    const jobs = await Job
      .find({
        companyId: record.id
      })
      .sort({
        createdAt: "desc"
      });

    const city = await City.findOne({
      _id: record.city
    })

    for (const item of jobs) {
      const itemFinal = {
        id: item.id,
        companyLogo: record.logo,
        title: item.title,
        companyName: record.companyName,
        salaryMin: item.salaryMin,
        salaryMax: item.salaryMax,
        position: item.position,
        workingForm: item.workingForm,
        cityName: city?.name,
        technologies: item.technologies
      };

      dataFinal.push(itemFinal);
    }

    res.json({
      code: "success",
      message: "Thành công!",
      companyDetail: companyDetail,
      jobs: dataFinal,
    })
  } catch (error) {
    console.log(error);
    res.json({
      code: "error",
      message: "Dữ liệu không hợp lệ!"
    })
  }
}
```

Đoạn code này là **hàm xử lý API lấy thông tin chi tiết của một công ty** ở phía backend.

Đoạn code trên xây dựng chức năng lấy chi tiết công ty. Khi frontend gửi yêu cầu đến API chi tiết công ty, backend sẽ lấy id từ đường dẫn thông qua req.params.id, sau đó tìm kiếm công ty tương ứng trong collection AccountCompany. Nếu không tìm thấy dữ liệu, hệ thống trả về thông báo lỗi. Nếu tìm thấy, backend sẽ gom các thông tin cần thiết của công ty như logo, tên công ty, địa chỉ, mô hình công ty, số lượng nhân viên, thời gian làm việc, chính sách làm thêm và mô tả công ty vào biến companyDetail.

Sau đó, hệ thống tiếp tục lấy **danh sách công việc thuộc công ty đó**:

```
const jobs = await Job.find({
  companyId: record.id
}).sort({
  createdAt: "desc"
});
```

Đoạn này dùng để tìm các job có companyId trùng với công ty hiện tại, đồng thời sắp xếp job mới nhất lên trước.

Tiếp theo:

```
const city = await City.findOne({
  _id: record.city
});
```

Dùng để lấy thông tin thành phố của công ty.

Sau đó code duyệt qua từng job:

```
for (const item of jobs) {
```

Mỗi job sẽ được format lại thành một object mới gồm các thông tin cần hiển thị ngoài frontend như:

- id công việc
- logo công ty
- tiêu đề công việc
- tên công ty
- mức lương min/max
- vị trí tuyển dụng
- hình thức làm việc
- tên thành phố
- công nghệ yêu cầu

Cuối cùng backend trả dữ liệu về frontend:

```
res.json({  
  code: "success",  
  message: "Thành công!",  
  companyDetail: companyDetail,  
  jobs: dataFinal  
});
```

Nghĩa là API trả về 2 phần chính:

companyDetail: thông tin chi tiết công ty

jobs: danh sách công việc của công ty đó

2.3.3. Kết quả

The screenshot displays the 28.ITJobs website interface. At the top, a dark blue navigation bar contains the logo '28.ITJobs' and links for 'Việc Làm IT', 'Top Công Ty IT', 'Nhà Tuyển Dụng', and 'Đăng Nhập / Đăng Ký'. The main content area features a card for 'VietQR' with its logo, location 'Cầu giấy', and details: 'Mô hình công ty: Vừa', 'Quy mô công ty: Vừa', 'Thời gian làm việc: Thứ 2,4,6', and 'Làm việc ngoài giờ: Không'. Below this, a section titled 'Công ty có 3 việc làm' lists three job openings, each with a VietQR logo, job title, salary range, experience level, office location, and city.

Job Title	Salary Range	Experience Level	Office Location	City
Fresher Frontend	5.000.000\$ - 20.000.000\$	Fresher	Tại văn phòng	Hà Nội
intern backend	2.000.000\$ - 16.000.000\$	Intern	Tại văn phòng	Hà Nội
Lập trình viên Frontend	2.000.000\$ - 15.000.000\$	Intern	Tại văn phòng	Hà Nội

Chi tiết công ty sẽ hiển thị khi chọn 1 công ty tương ứng.

2.4. Trang quản lý CV

2.4.1. Xử lý bên phía Frontend

```
useEffect(() => {
  fetch(`${process.env.NEXT_PUBLIC_API_URL}/company/cv/list`, {
    method: "GET",
    credentials: "include", // Gửi kèm cookie
  })
    .then(res => res.json())
    .then(data => {
      if(data.code == "success") {
        setCVList(data.cvs);
      }
    })
  }, []);
```

Đoạn code trên dùng để gọi API lấy danh sách CV đã ứng tuyển vào công ty. Khi component được tải, frontend gửi yêu cầu GET đến API /company/cv/list, đồng thời gửi kèm cookie thông qua credentials: "include" để backend xác thực tài khoản công ty đang đăng nhập. Nếu API trả về thành công, danh sách CV sẽ được lưu vào state cvList để hiển thị trên giao diện.

```
{cvList.map((item) => {
  const position = positionList.find(pos => pos.value == item.jobPosition);
  const workingForm = workingFormList.find(work => work.value == item.jobWorkingForm);
  const status = cvStatusList.find(st => st.value == item.status);
```

Đoạn code trên dùng để duyệt qua danh sách CV đã lấy từ API. Với mỗi CV, hệ thống sẽ đối chiếu các giá trị như vị trí ứng tuyển, hình thức làm việc và trạng thái CV với các danh sách cấu hình tương ứng. Việc này giúp chuyển dữ liệu dạng mã hoặc giá trị lưu trong database thành nội dung dễ hiểu để hiển thị trên giao diện, ví dụ như “Frontend”, “Toàn thời gian” hoặc “Chờ duyệt”.

```
export const cvStatusList = [
  {
    label: "Chưa duyệt",
    value: "initial",
    color: "□ #121212"
  },
  {
    label: "Đã duyệt",
    value: "approved",
    color: "■ #9FD87C"
  },
  {
    label: "Từ chối",
    value: "rejected",
    color: "■ #FF5100"
  },
];
```

Đoạn code trên dùng để định nghĩa danh sách các trạng thái của CV trong hệ thống. Mỗi trạng thái bao gồm tên hiển thị, giá trị xử lý và màu sắc tương ứng. Khi hiển thị danh sách CV, frontend sẽ dựa vào giá trị status của từng CV để tìm trạng thái phù hợp trong cvStatusList, từ đó hiển thị nội dung như “Chưa duyệt”, “Đã duyệt”, “Từ chối” kèm màu sắc trực quan trên giao diện.

2.4.2. Xử lý bên phía backend

```
router.get(
  "/cv/list",
  authMiddleware.verifyTokenCompany,
  companyController.listCV
)
```

Đoạn code trên khai báo route /cv/list dùng để lấy danh sách CV của công ty. Trước khi xử lý yêu cầu, hệ thống sử dụng middleware verifyTokenCompany để xác thực tài khoản công ty đang đăng nhập. Nếu xác thực thành công, request sẽ được chuyển đến hàm companyController.listCV để truy vấn và trả về danh sách CV tương ứng. Cách xử lý này giúp bảo vệ dữ liệu CV, đảm bảo chỉ công ty đã đăng nhập mới có quyền xem danh sách ứng viên.

```

export const listCV = async (req: AccountRequest, res: Response) => {
  try {
    const companyId = req.account.id;

    const jobList = await Job.find({
      companyId: companyId
    })
    const jobIdList = jobList.map(item => item.id);

    const cvs = await CV
      .find({
        jobId: { $in: jobIdList }
      })
      .sort({
        createdAt: "desc"
      });

    const dataFinal = [];

    for (const item of cvs) {
      const jobDetail = await Job.findOne({
        _id: item.jobId
      })

      if(jobDetail) {
        const itemFinal = {
          id: item.id,
          jobTitle: jobDetail.title,
          fullName: item.fullName,
          email: item.email,
          phone: item.phone,
          jobSalaryMin: jobDetail.salaryMin,
          jobSalaryMax: jobDetail.salaryMax,
          jobPosition: jobDetail.position,
          jobWorkingForm: jobDetail.workingForm,
          viewed: item.viewed,
          status: item.status,
        };

        dataFinal.push(itemFinal);
      }
    }

    res.json({
      code: "success",
      message: "Thành công!",
      cvs: dataFinal
    })
  } catch (error) {
    console.log(error);
    res.json({
      code: "error",
      message: "Dữ liệu không hợp lệ!"
    })
  }
}

```

Đây là hàm xử lý API lấy danh sách CV của công ty. Backend lấy ID công ty từ tài khoản đã đăng nhập, sau đó truy vấn danh sách công việc thuộc công ty đó. Từ danh sách công việc này, hệ thống tiếp tục tìm các CV có jobId tương ứng. Mỗi CV được kết hợp thêm thông tin công việc như tên job, mức lương, vị trí và hình thức làm việc, sau đó được đưa vào mảng dataFinal để trả về cho frontend. Nhờ vậy, công ty có thể xem danh sách ứng viên đã ứng tuyển vào các công việc của mình.

2.4.3. Kết quả

Quản lý CV

Frontend Engineer
(ReactJS)

Ứng viên: **Lê Văn A**

✉ levana@gmail.com

☎ 0123456789

1.000\$ - 1.500\$

👤 Fresher

📁 Tại văn phòng

🔴 Chưa xem

✅ Chưa duyệt

XemDuyệt

Từ chốiXóa

 Trang 1 ▾

Danh sách CV được quản lý sẽ hiện ra cho Nhà tuyển dụng

3. Kết quả đạt được

Trong tuần này, em đã hoàn thiện được trang danh sách công ty. Danh sách công ty đã được lấy từ API và hiển thị lên giao diện, giúp người dùng có thể xem các công ty đang có trên hệ thống.

Em cũng đã bổ sung thành công chức năng phân trang cho danh sách công ty. Khi số lượng công ty nhiều, hệ thống có thể chia dữ liệu thành nhiều trang, giúp giao diện gọn gàng hơn và người dùng dễ theo dõi hơn. Việc chọn trang được xử lý thông qua thẻ select, mỗi lần người dùng đổi trang thì frontend sẽ gọi lại API để lấy danh sách công ty tương ứng.

Bên cạnh đó, em đã xây dựng được trang chi tiết công ty. Khi người dùng chọn một công ty cụ thể, hệ thống sẽ chuyển sang trang chi tiết và hiển thị đầy đủ thông tin của công ty đó. Ngoài thông tin công ty, trang này còn hiển thị danh sách các công việc đang được đăng bởi công ty, giúp người dùng có thể xem thêm các vị trí tuyển dụng liên quan.

Ngoài ra, em đã xây dựng được trang quản lý CV cho nhà tuyển dụng. Nhà tuyển dụng sau khi đăng nhập có thể xem danh sách CV mà ứng viên đã gửi vào các công việc của công ty mình. Danh sách CV hiển thị các thông tin cần thiết như tên công việc, họ tên ứng viên, email, số điện thoại, mức lương, vị trí, hình thức làm việc, trạng thái đã xem và trạng thái xử lý hồ sơ.

Nhìn chung, các chức năng trong tuần này đã giúp hoàn thiện thêm luồng làm việc của website tuyển dụng, bao gồm phía người dùng xem công ty và phía nhà tuyển dụng quản lý CV ứng tuyển.

4. Khó khăn gặp phải

Trong quá trình thực hiện trang danh sách công ty, em gặp khó khăn khi xử lý dữ liệu trả về từ API và truyền dữ liệu sang component con. Vì mỗi công ty có nhiều thông tin khác nhau nên cần đảm bảo dữ liệu được truyền đầy đủ và đúng định dạng cho component `CardCompanyItem`.

Ở chức năng phân trang, khó khăn chính là việc đồng bộ giữa frontend và backend. Frontend cần lưu đúng trang hiện tại, còn backend cần nhận đúng tham số page, tính toán chính xác skip, limitItems và totalPages. Nếu một trong các giá trị này sai thì dữ liệu hiển thị sẽ không đúng với trang người dùng chọn.

Đối với trang chi tiết công ty, dữ liệu cần hiển thị không chỉ nằm trong bảng công ty mà còn liên quan đến các bảng khác như công việc và thành phố. Vì vậy, backend phải truy vấn thêm danh sách công việc theo companyId và lấy thêm thông tin thành phố để trả về dữ liệu đầy đủ cho frontend.

Một khó khăn khác là phần mô tả công ty được lưu dưới dạng HTML. Nếu hiển thị bằng cách thông thường, nội dung có thể bị hiện nguyên thể HTML thay vì được định dạng đúng. Do đó, frontend cần sử dụng `dangerouslySetInnerHTML` để render phần mô tả, đồng thời phải lưu ý vấn đề an toàn dữ liệu khi hiển thị HTML.

Với trang quản lý CV, khó khăn nằm ở việc xác thực tài khoản công ty. API lấy danh sách CV chỉ được phép truy cập khi công ty đã đăng nhập, nên frontend phải gửi kèm cookie thông qua credentials: "include", còn backend phải kiểm tra token bằng middleware `verifyTokenCompany`.

Ngoài ra, khi hiển thị danh sách CV, dữ liệu trong database thường được lưu dưới dạng giá trị xử lý như initial, approved, rejected, hoặc các mã vị trí, mã hình thức làm việc. Vì vậy, frontend cần đối chiếu các giá trị này với danh sách cấu hình tương ứng để hiển thị thành nội dung dễ hiểu cho người dùng.

5. Hướng xử lý

Để xử lý trang danh sách công ty, em sử dụng useState để lưu danh sách công ty và useEffect để gọi API khi component được render. Sau khi API trả về dữ liệu thành công, em lưu danh sách công ty vào state và dùng hàm map() để hiển thị từng công ty thông qua component CardCompanyItem.

Với chức năng phân trang, em bổ sung thêm state page để lưu trang hiện tại và totalPages để lưu tổng số trang. Khi người dùng chọn trang khác, hàm handlePagination sẽ lấy giá trị trang từ thẻ select, chuyển giá trị đó sang kiểu số và cập nhật lại state page. Khi page thay đổi, useEffect sẽ tự động gọi lại API để lấy dữ liệu mới.

Ở phía backend, em xử lý phân trang bằng cách đặt mặc định page = 1, sau đó kiểm tra nếu request có truyền page thì lấy giá trị đó từ req.query.page. Tiếp theo, em tính số bản ghi cần bỏ qua bằng công thức:

```
const skip = (page - 1) * limitItems;
```

Sau đó, backend dùng countDocuments() để đếm tổng số bản ghi, dùng Math.ceil() để tính tổng số trang và trả totalPages về cho frontend.

Đối với trang chi tiết công ty, em xây dựng route riêng để lấy thông tin chi tiết công ty. Backend lấy id từ request, tìm công ty tương ứng trong collection AccountCompany, sau đó gom các thông tin cần thiết vào object companyDetail. Đồng thời, backend truy vấn thêm danh sách công việc thuộc công ty đó và định dạng lại dữ liệu trước khi trả về frontend.

Ở frontend, em lấy slug từ URL, gọi API chi tiết công ty và lưu dữ liệu trả về vào các biến companyDetail và jobList. Phần mô tả công ty được hiển thị bằng dangerouslySetInnerHTML để nội dung HTML được render đúng định dạng.

Với trang quản lý CV, em xử lý bằng cách gọi API /company/cv/list khi component được tải. Request được gửi kèm cookie bằng credentials: "include" để backend xác thực tài khoản công ty. Nếu API trả về thành công, danh sách CV được lưu vào state cvList.

Ở backend, em sử dụng middleware verifyTokenCompany để kiểm tra token công ty trước khi cho phép truy cập API lấy danh sách CV. Sau khi xác thực thành công, backend lấy companyId từ req.account.id, tìm danh sách công việc thuộc công ty đó, sau

đó lấy các CV có jobId nằm trong danh sách công việc này. Cuối cùng, dữ liệu CV được kết hợp thêm thông tin công việc và trả về frontend.

6. Kế hoạch tuần tiếp theo

Trong tuần tiếp theo, em dự kiến tiếp tục hoàn thiện các chức năng liên quan đến quản lý CV trong website tuyển dụng việc làm, tập trung vào cả phía nhà tuyển dụng và phía ứng viên.

Trước hết, em sẽ xây dựng trang chi tiết CV dành cho nhà tuyển dụng. Khi nhà tuyển dụng chọn một CV trong danh sách quản lý, hệ thống sẽ chuyển sang trang chi tiết để hiển thị đầy đủ thông tin ứng viên, thông tin công việc đã ứng tuyển và file CV mà ứng viên đã gửi. Chức năng này giúp nhà tuyển dụng dễ dàng kiểm tra hồ sơ của từng ứng viên trước khi đưa ra quyết định xử lý.

Tiếp theo, em sẽ phát triển tính năng duyệt và từ chối CV trên trang quản lý CV. Nhà tuyển dụng có thể cập nhật trạng thái hồ sơ từ “Chưa duyệt” sang “Đã duyệt” hoặc “Từ chối”. Khi trạng thái thay đổi, dữ liệu sẽ được cập nhật lại trong cơ sở dữ liệu và hiển thị trực quan trên giao diện để nhà tuyển dụng dễ theo dõi quá trình xử lý hồ sơ.

Bên cạnh đó, em sẽ bổ sung tính năng xóa CV trong trang quản lý CV. Chức năng này cho phép nhà tuyển dụng xóa những hồ sơ không còn cần thiết khỏi danh sách quản lý. Khi thực hiện xóa, hệ thống sẽ gửi yêu cầu đến backend để xử lý và cập nhật lại danh sách CV trên giao diện.

Ngoài phần dành cho nhà tuyển dụng, em cũng dự kiến xây dựng trang quản lý CV đã gửi dành cho ứng viên. Trang này giúp ứng viên xem lại các công việc mà mình đã ứng tuyển, bao gồm thông tin công việc, công ty, thời gian gửi CV và trạng thái xử lý hồ sơ. Nhờ đó, ứng viên có thể theo dõi quá trình ứng tuyển của mình trong hệ thống.

Cuối cùng, em sẽ kiểm tra lại toàn bộ các chức năng liên quan đến CV như hiển thị danh sách CV, xem chi tiết CV, duyệt CV, từ chối CV, xóa CV và quản lý CV đã gửi để đảm bảo dữ liệu hiển thị chính xác, giao diện dễ sử dụng và các API hoạt động ổn định.